SWINBURNE UNIVERSITY OF TECHNOLOGY
SCHOOL OF SCIENCE, COMPUTING
AND ENGINEERING TECHNOLOGIES

PROJECT PROPOSAL

Focus Mode App — A Serverless, AI-Augmented Productivity & Well-being Platform on AWS

AWS First Cloud Journey (FCJ) Internship 2026

Full Name	Student ID
Le Hoang Triet Thong	104171146

Ho Chi Minh City, 2026

Table of Contents

Executive Summary

Knowledge workers and students increasingly struggle to sustain deep focus in an environment engineered for distraction. Conventional Pomodoro timers address only the mechanics of time-boxing; they do nothing about the two factors that actually erode a work session: a distracting environment and the invisible emotional cost of the work itself. **Focus Mode App** is a cloud-native productivity and well-being platform that closes this gap. It combines an immersive focus timer with ambient sound, an AI task assistant, automatic post-session emotion detection, and emotion-aware content recommendations — all delivered through a fully serverless architecture on Amazon Web Services (AWS) and Supabase.

This proposal presents the solution's business rationale, its technical architecture, the implementation approach, a realistic delivery timeline, a detailed budget grounded in actual AWS pricing, a risk assessment, and the expected outcomes. Unlike a purely hypothetical proposal, every architectural decision and cost figure here is drawn from a working reference implementation: the system described has been built and deployed to a live production environment, so the numbers are measured rather than guessed.

Solution overview. A single-page web application (Nuxt 3 / Vue 3), hosted on AWS Amplify, talks to a Supabase Backend-as-a-Service (PostgreSQL, Auth, and the `pgvector` extension) for all core CRUD and authentication. Five AI and media capabilities run as independent AWS Lambda functions behind an Amazon API Gateway HTTP API, using Amazon Bedrock for large-language-model reasoning (a Claude Haiku 4.5 task agent) and multilingual text embeddings (Cohere Embed Multilingual v3). A sixth Lambda manages ambient-audio uploads to Amazon S3. Emotion detection runs a quantized DistilBERT model packaged directly inside its Lambda.

Key features:

- Wall-clock-accurate Pomodoro focus timer with ambient audio streamed from S3.
- A conversational AI task assistant (Amazon Bedrock Agent) that creates, lists, updates and deletes tasks from natural-language requests.
- Automatic emotion detection from a post-session journal entry (five labels).
- Retrieval-augmented (RAG) content recommendations matched to the detected emotion.
- Task management, a focus-history heatmap, downloadable worklog reports, an in-app user guide, and a feedback channel.
- An admin CMS for user approval, media/embedding management, ambient-sound management, and feedback triage.

Business benefits and ROI. The architecture is deliberately free-tier-first. At a representative pilot scale of ten users the measured operating cost is approximately **$0.97 per month** (about **$12 per year**), rising to no more than **$1.51 per month** in the conservative worst case. Because the only unavoidable fixed costs are Amazon Bedrock (usage-based) and a single Route 53 hosted zone ($0.50/month), the platform reaches positive ROI almost immediately:

even a modest improvement in a single knowledge worker's focused output dwarfs an annual infrastructure bill smaller than the price of one lunch.

Investment and timeline. The reference implementation was delivered by a single intern developer across a phased schedule of roughly nine weeks (backend foundation → frontend and auth → core focus/task features → cloud AI services → administration, hardening and documentation). Infrastructure investment is effectively zero; the dominant cost is developer time.

Success metrics and expected outcomes. Success is measured by a working end-to-end focus loop (sign-in → plan tasks → focus with ambient sound → journal → emotion → recommendation → report), by all six AI/media Lambdas being live and verified end-to-end, and by keeping monthly cost under $2 at pilot scale. These targets have been met in the reference build, which was internally assessed at 8.5/10 against a High-Distinction rubric. The strategic outcome is a reusable, well-architected serverless pattern — managed BaaS for data plus event-driven AWS compute for AI — that can be extended to other emotion-aware or agentic applications.

1 Problem Statement

1.1 Current Situation

The modern study-and-work environment is saturated with interruptions. Instant messaging, notifications, social feeds and context-switching between tools fragment attention into short bursts. The cost of a single interruption is not just the seconds it lasts: research on attention consistently shows that returning to a demanding task after an interruption takes many minutes of re-focus, and that frequent context switching measurably reduces the quality and volume of deep work. For students and knowledge workers whose output depends on sustained concentration, this is a direct productivity tax paid every day.

At the same time, the tools people reach for to fight distraction are shallow. The typical Pomodoro or "focus timer" app does one thing: it counts down. It does not shape the working environment, it does not understand what the user is working on, and — crucially — it is blind to the user's psychological state. Burnout, frustration and loss of motivation build up silently across many sessions, and a bare countdown timer offers no way to notice or respond to them.

1.2 Key Challenges (Pain Points)

- **Environmental distraction.** A plain timer does nothing to create an immersive, low-distraction environment. Users still see a bright, busy screen and hear whatever is around them.
- **Cognitive overhead of planning.** Turning a vague intention ("prepare my CV", "revise for the exam") into a concrete, ordered task list is itself effortful, and that friction happens exactly when the user is trying to start working.
- **Invisible emotional cost.** Most tools ask users to self-rate their mood on a slider — a step users skip or answer carelessly. The emotional signal that would let the tool intervene (encourage a break, surface calming content) is never captured reliably.
- **No supportive feedback loop.** When a user finishes a hard, draining session, nothing responds to that. There is no mechanism that offers the right encouragement or content at the moment it would actually help.
- **Fragmented history and reporting.** Focus data, task completion and mood live in separate places, if they are recorded at all, so users cannot easily see patterns or review a day's work.
- **Cost and operational burden for the builder.** For an individual developer or a small team, standing up the servers, databases and AI infrastructure to support these features traditionally means real recurring cost and real operations work — a barrier that keeps such tools from being built at all.

1.3 Stakeholders Affected

- **End users (students and knowledge workers).** The primary beneficiaries. They want to enter deep focus quickly, be supported emotionally, and review their progress —

without manual bookkeeping.

- **Administrators / content curators.** They approve users, curate the knowledge base of supportive content (quotes, sutras, lecture transcripts, videos), manage ambient audio, and read user feedback. They need a simple, role-gated web console rather than direct database access.
- **The developer / operator.** Needs the whole system to run within free tiers, deploy without heavyweight operations, and scale to zero when idle so that an idle user base costs almost nothing.

1.4 Business Consequences of Inaction

Without a solution, the productivity tax of fragmented attention continues unchecked, and the well-being dimension of focused work stays completely unaddressed by the existing crop of timer apps. From a builder's perspective, the alternative architectures are worse: a traditional always-on server incurs fixed monthly cost regardless of usage, requires patching and monitoring, and does not scale gracefully. Doing nothing therefore means either accepting an inferior user experience or accepting an operating model that is too expensive and too operationally heavy to justify for a small user base.

1.5 Market Opportunity

Productivity and focus applications are a large and growing category, and the differentiator that this proposal targets — *emotion-aware, AI-assisted* focus rather than a bare timer — is still uncommon. The opportunity is to combine three modern capabilities that are now cheap to access individually:
(1) managed Backend-as-a-Service for data and auth
(2) serverless compute that scales to Zero
(3) on-demand large-language-model and embedding APIs. Assembled correctly, these let a single developer ship a differentiated, AI-rich product at near-zero fixed cost — an opportunity that did not exist a few years ago and that this proposal demonstrates is now realistic.

Problem in one sentence: existing focus tools count time but ignore the environment, the planning burden, and the user's emotional state — and building a tool that addresses all three has, until recently, been too costly and operationally heavy for an individual to attempt.

2 Solution Architecture

2.1 Architecture Overview

Focus Mode App uses an **event-driven, serverless, cloud-native** architecture built on two complementary platforms. **Supabase** provides the managed data plane — PostgreSQL, authentication, and the `pgvector` vector store — so that no database server has to be operated. **AWS** provides the compute and AI plane: a static web host (Amplify), DNS (Route 53), an HTTP API (API Gateway), six single-responsibility Lambda functions, and Amazon Bedrock for

language-model reasoning and embeddings. Each component has one job, scales independently, and stays within free-tier limits at pilot scale. Figure 1 shows the as-built architecture.

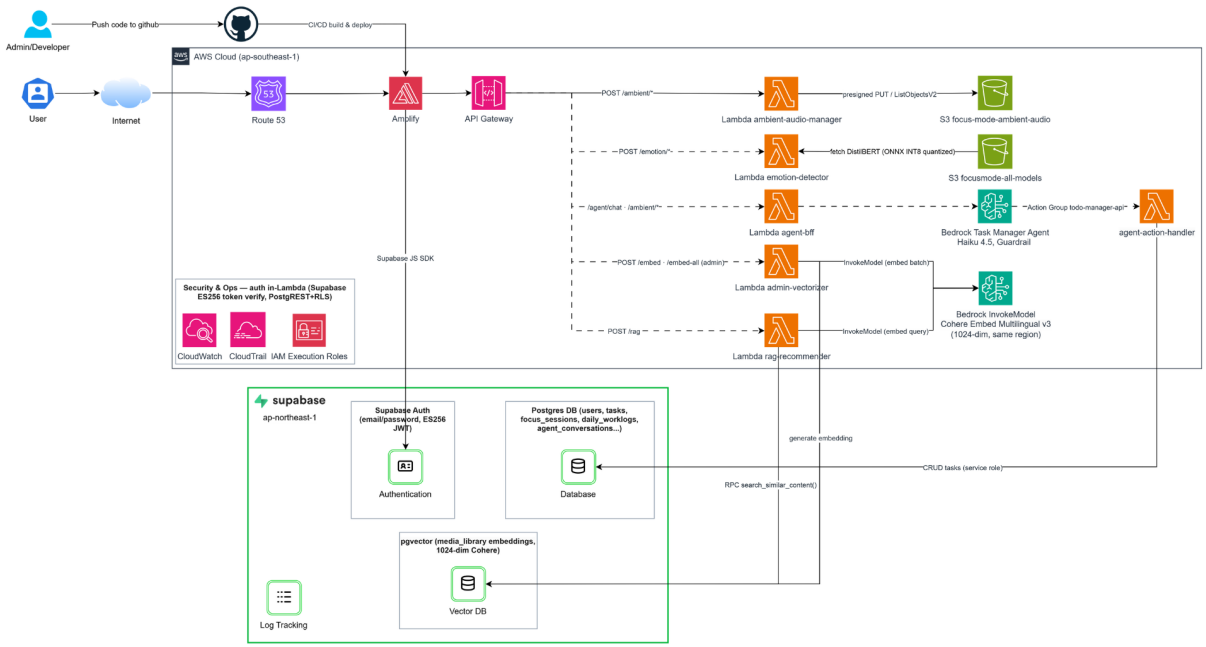

Figure 1: As-built architecture. The user's browser loads the Nuxt SPA from Amplify (via Route 53) and talks directly to Supabase for CRUD/auth; AI and media features are invoked through API Gateway to six Lambda functions, which call Amazon Bedrock and Supabase. Authentication is verified inside each Lambda against Supabase (ES256 + PostgREST + RLS).

2.2 AWS Services Used and Justification

Table 1 lists each service and why it was selected over the alternatives.

Service	Role in the system	Why this choice
AWS Amplify Hosting	Hosts the static Nuxt SPA; Git-based CI/CD	Zero-ops static hosting with a built-in build pipeline and generous free tier; no web server to manage.
Amazon Route 53	DNS for the custom domain	Native alias records to Amplify (free DNS queries); single hosted zone; integrates with the rest of AWS.
Amazon API Gateway (HTTP API)	Single public entry point for all Lambda routes	HTTP API is cheaper and lower-latency than REST API; ample free tier; simple route-to-Lambda integration.
AWS Lambda (×6)	Runs all AI/media logic, event-driven	Scales to zero (idle users cost nothing); per-request billing; 400,000 GB-seconds/month free *perpetually*.
Amazon Bedrock — Agent (Claude Haiku 4.5)	Conversational task assistant	Managed LLM with tool-use/orchestration and guardrails; Haiku 4.5 gives the highest requests-per-minute quota available on the account (50 RPM) at low cost.
Amazon Bedrock — Cohere Embed Multilingual v3	Text embeddings for RAG & vectorization	Available directly in `ap-southeast-1` (no cross-region call), 1024-dim, strong multilingual (incl. Vietnamese) quality; no inference profile needed.
Amazon S3	Ambient-audio files; ONNX model artifact	Cheap, durable object storage; presigned uploads keep credentials off the client.
Amazon CloudWatch / CloudTrail	Logs, metrics, audit trail	Built-in observability for every Lambda; perpetual free log tier covers pilot scale.
AWS IAM	Least-privilege execution roles	Scopes each Lambda to exactly the Bedrock/S3/logs actions it needs.

Table 1: AWS service selection and justification.

Why Supabase for the data plane. Rather than run PostgreSQL on AWS (RDS), the design delegates the database, authentication and vector store to Supabase's managed free tier (500 MB database, 50,000 monthly active users, `pgvector` pre-enabled). This removes an entire class of operational work (backups, connection pooling, auth flows, Row-Level-Security enforcement) and keeps the AWS side focused purely on stateless compute and AI.

2.3 Component Design

The six Lambda functions each have a single responsibility:

1. **`agent-bff`** — the "backend for frontend" for chat. It authenticates the caller, injects the current date and a per-user session namespace, and synchronously invokes the Bedrock Agent.
2. **`agent-action-handler`** — the Bedrock Agent's action group (`todo-manager-api`). Bedrock calls it to `create`/`list`/ `update`/`delete` tasks; it writes to Supabase, always scoped to the authenticated user.
3. **`emotion-detector`** — classifies journal text into one of five labels (*focused, stressed, exhausted, relaxed, unmotivated*) using a DistilBERT model exported to ONNX and quantized to INT8, packaged directly inside the Lambda.

4. **admin-vectorizer** — admin-only; generates Cohere embeddings for media-library items (single or batch) and stores them in pgvector.
5. **rag-recommender** — maps a detected emotion to a query, embeds it with Cohere, and runs a cosine-similarity search (search_similar_content()) to return the most relevant supportive content.
6. **ambient-audio-manager** — issues S3 presigned upload URLs and lists ambient-audio objects for the admin console.

2.4 Data Model

All persistent state lives in Supabase PostgreSQL. Table 2 summarizes the main tables; the media-library table additionally carries a 1024-dimension pgvector column with an ivfflat cosine index.

Table	Purpose
users	Mirrors auth.users; holds role and approval status (pending/approved/rejected).
tasks	To-do items: status, priority, review, immutable completed_at.
focus_sessions	Focus records: planned/actual duration, journal text, emotion label + confidence, ambient track.
daily_worklogs / daily_stats	Aggregated daily focus statistics.
media_library	RAG content (quote/sutra/video/article/audio) with a VECTOR(1024) embedding.
ambient_sounds	Admin-managed ambient tracks (S3 URL, active flag, order).
agent_conversations / agent_messages	Persisted AI chat history.
agent_daily_usage	Per-user daily call counter for rate limiting.
feedback	In-app user feedback (message, status new/read/resolved).

Table 2: Core data model (Supabase public schema).

2.5 Security Architecture

Security is layered and does not rely on the client:

- **Authentication.** Supabase Auth issues **ES256**-signed JWTs. Because API Gateway's built-in JWT authorizer supports only RS256, each Lambda verifies the token *in-function* by calling Supabase PostgREST with the caller's token; the database then validates the signature and applies Row-Level Security. Admin routes additionally check role = 'admin'.
- **Authorization at the data layer.** Row-Level Security (RLS) policies gate every table so a user can only read/write their own rows; the is_admin() helper backs admin-only policies. A BEFORE UPDATE trigger blocks a user from self-escalating their own role/status.
- **LLM safety.** The Bedrock Agent runs behind a **Guardrail** (prompt-attack filtering at HIGH, denied topics) and a hardened system prompt that treats user-supplied content as

data, not instructions. The action handler enforces a field whitelist and fail-closed user scoping to prevent confused-deputy and mass-assignment attacks; the per-user session namespace prevents session hijacking.

- **Least privilege.** Each Lambda's IAM execution role grants only the specific Bedrock/S3/logs actions it uses. Presigned S3 uploads keep bucket credentials off the browser.
- **Abuse controls (at demo scale).** A per-user daily call limit (`AGENT_DAILY_LIMIT`) and an input-size cap bound cost and abuse. Network-edge rate limiting / WAF is intentionally out of scope at pilot scale (see §6).

2.6 Scalability and Performance

Because compute is serverless, throughput scales automatically with demand and falls to zero cost when idle — the ideal profile for a bursty, per-user workload. Supabase's managed Postgres handles connection pooling and read scaling on the data side. The main performance considerations are Lambda cold starts (the emotion model adds a few seconds on a cold invocation but 1–2 seconds when warm) and the Bedrock Agent's requests-per-minute quota (Claude Haiku 4.5 provides 50 RPM, ample for pilot scale and raisable via Service Quotas). A known ceiling — long multi-step agent operations approaching the API Gateway integration timeout — is discussed as a risk in §6.

2.7 Representative Data Flows

AI task assistant: browser → API Gateway `/agent/chat` → `agent-bff` (auth + invoke) → Bedrock Agent (Claude Haiku 4.5 + Guardrail) → action group `agent-action-handler` → Supabase `tasks` → response streamed back to the chat UI.

Emotion-aware recommendation: on session end, the journal text is sent to `/emotion` → `emotion-detector` returns a label + confidence → the label is sent to `/rag` → `rag-recommender` embeds a query (Cohere) and runs `search_similar_content()` over `pgvector` → the top supportive items are shown to the user.

3 Technical Implementation

3.1 Implementation Phases and Deliverables

Implementation follows five incremental phases, each ending in a demonstrable deliverable:

1. **Backend foundation.** Provision Supabase; define the schema, triggers, and RLS via sequential SQL migrations. *Deliverable:* a secured database with a working sign-up/approval flow.
2. **Frontend and auth.** Scaffold the Nuxt 3 SPA; wire Supabase Auth, route middleware (`auth`/`admin`) and the app shell. *Deliverable:* authenticated navigation across the core pages.

3. **Core focus/task features.** Task CRUD with review, the wall-clock focus timer, ambient audio, the post-session journal, and the history heatmap. *Deliverable:* the complete offline-of-mind focus loop, cloud-persisted.
4. **Cloud AI services.** Implement and deploy the six Lambdas + Bedrock Agent; wire the frontend composables with authenticated calls. *Deliverable:* live agent chat, emotion detection, and RAG recommendations.
5. **Administration, hardening and documentation.** Admin CMS (users, media, ambient, feedback), the user guide, security hardening, and the full document set. *Deliverable:* a production-ready, documented system.

3.2 Technical Requirements

- **Compute:** six Python 3.12 Lambda functions; memory sized per function (512 MB for the emotion model, less for the others). No always-on server.
- **Storage:** Supabase PostgreSQL (500 MB free tier is ample at pilot scale); one S3 bucket for ambient audio and one artifact object for the ONNX model.
- **Network:** one API Gateway HTTP API with a route per Lambda; Route 53 alias records to Amplify; all traffic over HTTPS.
- **External APIs:** Amazon Bedrock (Claude Haiku 4.5 agent; Cohere Embed Multilingual v3) in `ap-southeast-1`.

3.3 Development Approach

The frontend is a Nuxt 3 single-page application (`ssr:false`) using Vue 3 Composition API, Pinia stores (`task`, `focus`, `user`) for state, a set of composables (`useAuth`, `useDataService`, `useAgentChat`, `useEmotionDetector`, `useRAG`, `useFeedback`, ...) as the service layer, and Tailwind CSS for styling. Database evolution is managed through numbered, idempotent SQL migrations applied in order. Backend logic is plain Python with minimal dependencies, favouring direct calls to Bedrock and Supabase over heavy frameworks.

3.4 Key Implementation Decisions

Several decisions materially shaped the build and are worth calling out:

- **Cloud-only instead of offline-first.** The original concept called for an offline-first client with a sync queue. This was deliberately dropped in favour of a simpler cloud-only model (all reads/writes go straight to Supabase), removing an entire class of conflict-resolution complexity for a single-user workload.
- **Claude Haiku 4.5 for the agent.** Selected because it offered the highest requests-per-minute quota available on the account (50 RPM, versus single-digit RPM for the alternatives), eliminating throttling on multi-turn conversations while remaining fast and inexpensive.

- **Cohere Embed Multilingual v3 for embeddings.** Chosen after verifying that Amazon Titan embeddings are not available in `ap-southeast-1`; Cohere is available in-region, is strongly multilingual, and needs no cross-region inference profile.
- **DistilBERT ONNX in-Lambda for emotion.** Packaging a quantized model directly in the function avoids a per-call Bedrock charge for a high-frequency, low-complexity classification and keeps the feature self-contained.
- **In-Lambda ES256 auth.** Because Supabase tokens are ES256 and API Gateway's JWT authorizer is RS256-only, verification was moved into each Lambda via PostgREST + RLS — a clean, defensible pattern rather than a workaround.

3.5 Testing Strategy

The intended testing pyramid is: unit tests for critical logic (Pinia stores, the agent action handler's field-whitelisting, RLS-gated writes), integration tests against a test Supabase instance, and a small set of end-to-end tests covering the core loop (login → task → focus → journal → report). During development, each Lambda was verified end-to-end with real tokens via `curl` and CloudWatch inspection, and the frontend was type-checked (`nuxi typecheck`) as the gate before every commit. Automated regression coverage is an acknowledged gap that is deliberately scheduled last for the pilot (see §6).

3.6 Deployment Plan and Rollback

The frontend deploys automatically: a push to the GitHub repository triggers an Amplify build (`nuxt generate`) and release. Lambdas and the Bedrock Agent are deployed via documented AWS CLI runbooks (package → upload → create/update function → attach route). **Rollback** for the agent is built in: agent changes are versioned and served through an alias, so reverting is a matter of repointing the `prod` alias to a previous version; frontend rollback is an Amplify redeploy of a prior commit; database changes are forward-only migrations, so a corrective migration is the rollback path.

3.7 Configuration Management

Runtime configuration is entirely environment-driven: the frontend reads `NUXT_PUBLIC_*` variables (API base URLs, app URL) at build time; each Lambda reads its Supabase URL/keys and model IDs from environment variables. Secrets are held as Lambda environment variables at pilot scale (a managed secrets store is noted as a future hardening step, not required for a demo-scale deployment).

4 Timeline & Milestones

4.1 Project Timeline

The project is delivered by a single intern developer over approximately nine working weeks, organised into the five phases from §3. Table 3 shows the schedule.

Week	Phase	Focus of work
1	Backend foundation	Supabase project, schema + migrations, auth trigger, RLS, user-approval flow.
2	Frontend & auth	Nuxt SPA scaffold, Supabase Auth, route middleware, app shell.
3–4	Core features	Task CRUD + review, wall-clock focus timer, ambient audio, journal, history heatmap, dashboard.
5–6	Cloud AI services	Deploy six Lambdas + Bedrock Agent + Guardrail; wire authenticated frontend composables; verify each end-to-end.
7–8	Administration & hardening	Admin CMS (users/media/ambient/feedback), user guide, security hardening, cost analysis.
9	Documentation & submission	Full document set, architecture diagram, presentation, this proposal; final review.

Table 3: Nine-week delivery schedule.

4.2 Key Milestones

- **M1 (end wk 1):** Secured database with working sign-up/approval.
- **M2 (end wk 2):** Authenticated SPA navigation.
- **M3 (end wk 4):** Complete focus loop, cloud-persisted — the internal MVP.
- **M4 (end wk 6):** All AI features live and verified end-to-end.
- **M5 (end wk 8):** Admin CMS + hardening complete; production live.
- **M6 (end wk 9):** Documentation and submission package complete.

4.3 Dependencies and Critical Path

The critical path runs schema → auth → core features → AI services → admin/hardening. The AI phase (M4) is the highest-risk link because it depends on external factors outside the codebase: Bedrock model access, RPM quotas, and correct IAM permissions. RAG additionally depends on the vectorizer (shared embedding dimension) and on the media library actually containing embedded content.

4.4 Resource Allocation and Buffer

Resourcing is a single developer, so phases run sequentially rather than in parallel; week 9 doubles as schedule buffer, absorbing spillover from the AI-integration phase (the most likely source of overrun). Because infrastructure is managed/serverless, there is no provisioning lead time to account for.

5 Budget Estimation

5.1 Costing Assumptions

Costs are modelled for a representative pilot of **ten users** (2 active, 3 occasional, 5 inactive), yielding roughly **52 focus sessions/month**. Within that, the assumed AI usage is: emotion detection once per session (52/month), agent chat on ∼40% of sessions at ∼3 turns each (∼63 turns/month), one RAG query per session (52/month), and ∼10 admin content embeddings/month. Token sizes are reasoned estimates (agent turn ≈ 1,500 input / 400 output tokens), not CloudWatch-measured figures, so a ±2× buffer is applied in the worst-case scenario. Prices were looked up from the official AWS pricing pages (July 2026), region `ap-southeast-1` where applicable.

5.2 Infrastructure Costs (Monthly)

Table 4 breaks down the monthly operating cost.

Service	$/month	Notes
Bedrock — Claude Haiku 4.5 (agent)	0.22	$1/$5 per 1M in/out tokens; usage-based; no free tier.
Bedrock — Cohere Embed Multilingual v3	<0.01	$0.10 per 1M input tokens; tiny at this scale.
AWS Lambda (×6)	0.00	Well within the perpetual free tier (1M req + 400k GB-s).
API Gateway (HTTP API)	0.00	Within free tier (∼hundreds of req/month).
Amazon S3 (ambient audio)	0.00	A few small MP3s; within free tier.
CloudWatch Logs	0.00	Within the perpetual 5 GB free tier.
AWS Amplify Hosting	0.00	<1 GB bandwidth + a few builds; within free tier.
Supabase (Postgres/Auth/pgvector)	0.00	Free tier (500 MB DB, 50k MAU).
Route 53 — domain registration	0.25	Real purchase, ∼$3/year amortized.
Route 53 — hosted zone	0.50	$0.50/zone/month, fixed, no free tier.
Route 53 — DNS queries	∼0.00	Alias records to Amplify are free.
Total (standard)	**≈0.97**	≈24,000 VND/month.
Total (worst case)	**≈1.51**	Free tiers expired + 2× token buffer.

Table 4: Monthly infrastructure cost at a 10-user pilot scale.

Two facts drive this budget. Only two cost groups have *no* free tier: Amazon Bedrock (usage-based) and the Route 53 hosted zone (a fixed $0.50/month regardless of traffic). Everything else fits inside free tiers at pilot scale — and Lambda's free tier is *perpetual*, while the API Gateway and Amplify free tiers apply only for the first 12 months of the AWS account.

5.3 Development and Operational Costs

The one-time development cost is developer time (approximately nine weeks of a single intern), not cash outlay. Ongoing operational cost beyond the table above is negligible: there are no servers to patch, no scaling to manage, and deployment is scripted. The only recurring cash cost is the sub-$2/month infrastructure bill plus the annual domain renewal.

5.4 ROI and Break-Even Analysis

At pilot scale the annual infrastructure cost is roughly **$12–18/year**. Framed against value, the break-even bar is trivially low: if the platform helps its users reclaim even a small amount of focused time per week, the value of that time vastly exceeds the cost. As an illustration (clearly a modelling assumption, not a measured result), if the tool saves each of ten users just 30 minutes of otherwise-lost time per week and that time is valued at even a modest hourly rate, the monthly value created is orders of magnitude larger than the ∼$1 monthly cost — so ROI is positive from the first month and break-even is effectively immediate. Should the product ever be commercialised, the same architecture supports a simple per-seat subscription whose marginal cost per user is a few cents of Bedrock usage, leaving a very high gross margin.

5.5 Cost-Optimization Strategies

- **Free-tier-first design:** every component except Bedrock and the hosted zone sits inside a free tier at pilot scale.
- **Scale-to-zero compute:** idle users incur no Lambda cost.
- **Model selection for cost:** Haiku (not a larger model) for the agent; a self-hosted quantized model for the high-frequency emotion task instead of a per-call API.
- **Input capping and batching:** capped input sizes bound token cost; the vectorizer embeds up to a batch of items in a single Bedrock call.
- **In-region services:** choosing Cohere in `ap-southeast-1` avoids cross-region data-transfer and latency.

6 Risk Assessment

6.1 Risk Matrix

Table 5 lists the principal risks with their likelihood, impact, and mitigation. Several of these are not hypothetical — they were encountered and handled during the reference build.

Risk	Likel.	Impact	Mitigation / contingency
Bedrock request throttling (RPM quota)	Med	Med	Use Haiku 4.5 (50 RPM, highest available); request a quota increase via Service Quotas if needed.
Long agent operations time out ("Service Unavailable")	Med	Med	Known API Gateway ∼30s integration ceiling on large multi-step requests; contingency is to batch tool calls or move to an async invocation model.
Free-tier expiry after 12 months (API GW, Amplify)	High	Low	Cost rises only marginally (cents); budgeted in the worst-case column.
Embedding truncation (>2000 chars)	Med	Low	Cohere hard-limits input; long items embed on a prefix. Contingency: content chunking (`media_chunks`).
No automated test coverage yet	High	Med	Manual end-to-end verification + type-checking today; add unit/E2E tests on the riskiest paths first.
Single-region / vendor dependence	Low	Med	Serverless design is portable; Supabase is standard Postgres; models are swappable behind the Lambda boundary.
Secrets stored as plain env vars	Low	Med	Acceptable at demo scale; migrate to a managed secrets store before any real multi-tenant launch.
Manual deploy (no IaC/CI for Lambdas)	Med	Low	Deployment runbooks are documented and repeatable; adopt SAM/CDK before scaling the team.

Table 5: Risk matrix (Likelihood / Impact / Mitigation).

6.2 Mitigation and Contingency Summary

The design's biggest structural protection is that each risk is contained behind a clear boundary: models can be swapped behind a Lambda, the data layer is portable standard Postgres, and the frontend is a static bundle. The most material *functional* risk — the agent timing out on very large batch requests — has a clear, understood remedy (batch the tool calls or invoke asynchronously) and does not affect normal single-action use.

6.3 Monitoring and Escalation

CloudWatch logs and metrics on every Lambda, plus CloudTrail for an audit trail, provide the visibility to detect throttling, errors and cost anomalies. At pilot scale, escalation is informal (the operator reviews logs and AWS Cost Explorer); a production launch would add alarms on error rate and spend.

7 Expected Outcomes

7.1 Success Metrics

- **Functional:** the complete focus loop works end-to-end, and all six AI/media Lambdas are live and verified (both achieved in the reference build).
- **Cost:** monthly operating cost stays under $2 at pilot scale (achieved: ≈$0.97).

- **Quality:** the frontend type-checks cleanly and each Lambda is verified via real-token end-to-end tests (achieved).
- **Assessment:** the reference build was internally assessed at **8.5/10** against a High-Distinction rubric.

7.2 Short-Term Benefits (0–6 months)

Users get an immersive, emotion-aware focus tool at essentially no infrastructure cost; the operator gets a live, documented, well-architected system that demonstrates managed BaaS + serverless AWS + Bedrock working together.

7.3 Medium-Term Benefits (6–18 months)

The architecture is a reusable template. Closing the acknowledged gaps — automated tests, content chunking for long transcripts, and an async path for large agent operations — would move the platform from a strong pilot to a genuinely production-grade product.

7.4 Long-Term Value (18+ months)

The pattern generalises well beyond this app: any product that needs managed data, scale-to-zero compute, and on-demand LLM/embedding capability can reuse it. Emotion-aware interaction and agentic task management are broadly applicable strategic capabilities. If commercialised, the near-zero marginal cost per user makes a high-margin subscription model viable.

7.5 User Experience Improvements

Compared with a conventional timer, the platform reduces the friction of planning (the AI assistant turns intentions into ordered tasks), makes the working environment immersive (ambient audio, focused UI), captures emotional signal automatically (no mood sliders), and closes the loop with supportive, relevant content — a qualitatively different experience from "a countdown on a screen".

Appendices

.1 Appendix A — Technical Specifications

- **Frontend:** Nuxt 3 (`ssr:false`), Vue 3 Composition API, Pinia, Tailwind CSS; deployed via `nuxt generate` to AWS Amplify.
- **Backend (data):** Supabase — PostgreSQL, Auth (ES256 JWT), `pgvector`; access governed by Row-Level Security and `is_admin()`.
- **Backend (compute):** six Python 3.12 AWS Lambda functions behind one API Gateway HTTP API; per-function least-privilege IAM roles.
- **AI models (Amazon Bedrock, `ap-southeast-1`):** Claude Haiku 4.5 (agent, global cross-region inference profile, 50 RPM, behind a Guardrail); Cohere Embed Multilingual v3 (1024-dim embeddings). Emotion: DistilBERT exported to ONNX, INT8-quantized, packaged in-Lambda; five labels (focused, stressed, exhausted, relaxed, unmotivated).
- **API routes:** `/agent/chat`, `/emotion`, `/rag`, `/embed` & `/embed-all` (admin), `/ambient/*`.
- **Regions:** AWS `ap-southeast-1` (Singapore); Supabase `ap-northeast-1`.

.2 Appendix B — Cost Calculation Detail

The per-service monthly figures in Table 4 derive from the usage model in §5.1. Worked examples for the two paid groups:

- **Agent chat (Bedrock Haiku 4.5):** 63 turns × 1,500 input tokens = 94,500 tokens ≈ 0.0945M × \$1.00 = \$0.0945 (input); 63 × 400 output = 25,200 tokens ≈ 0.0252M × \$5.00 = \$0.126 (output); total ≈ **\$0.22/month**.
- **Embeddings (Cohere Embed):** ∼52 RAG queries + ∼10 admin embeds, a few thousand tokens total × \$0.10 per 1M = well under one cent/month.

All other services fall within free tiers at this scale (see Table 4 notes). Figures are reasoned estimates, not production-measured; a one-month live run reconciled against AWS Cost Explorer would refine them.

.3 Appendix C — Architecture Diagram

See Figure 1 in §2.1 for the full as-built architecture diagram (rendered at full width). It shows the end-to-end request flow from the user's browser through Route 53, Amplify, and API Gateway to the six Lambdas, Amazon Bedrock, Amazon S3, and the Supabase data plane, together with the security/operations layer (CloudWatch, CloudTrail, IAM, in-Lambda ES256 auth).

.4 Appendix D — References

[1] Amazon Web Services — Bedrock pricing. https://aws.amazon.com/bedrock/pricing/

[2] Amazon Web Services — Lambda pricing (perpetual free tier). https://aws.amazon.com/lambda/pricing/

[3] Amazon Web Services — API Gateway pricing. https://aws.amazon.com/api-gateway/pricing/

[4] Amazon Web Services — Amplify Hosting pricing. https://aws.amazon.com/amplify/pricing/

[5] Amazon Web Services — Route 53 pricing. https://aws.amazon.com/route53/pricing/

[6] AWS Well-Architected Framework. https://aws.amazon.com/architecture/well-architected/

[7] Supabase documentation — Auth, PostgreSQL, and `pgvector`. https://supabase.com/docs

[8] Cohere Embed Multilingual v3 (via Amazon Bedrock) — model documentation.

.5 Appendix E — Showcase Links

[1] Website link: https://focusmode.click/

[2] Presentation Slides: https://canva.link/1a3sdbjvlyq0suq